AURIX

Microcontroller Infineon Software Architecture

User's Manual

User Mode / Supervisory Mode Implementation

Release v1.0

2016-08

Microcontrollers

Edition 2016-08

Published by
Infineon Technologies AG
81726 Munich, Germany

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (**www.infineon.com**).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Document Change History		
Date	**Version**	**Change Description**
2016.08.08	1.0	Updated based on review comments.
2016.08.07	0.2	Updated based on new method for user 0 mode support.
2015.10.27	0.1	Initial Version

Table of Contents **Page**

List of Figures **Page**

List of Tables **Page**

1 Introduction

This User Manual provides information regarding the implementation of MCAL drivers to support execution in I/O Privilege Level (User-0 /User-1/Supervisor) of the CPU.

1.1 Abbreviations

Table 1 Abbreviations used in this document

Abbreviation	Explanation
API	Application Programming Interface
MCAL	Micro Controller Abstraction Layer
SV	Supervisory

1.2 References

This section lists down all relevant documents for this User Manual.

1. All MCAL Drivers User Manual.

2 Previous Implementation

MCAL driver APIs can be executed when the CPU is in either User-1 or Supervisor Mode. This can be configured via precompile switch - IFX_MCAL_RUN_MODE_DEFINE.

- IFX_MCAL_RUN_MODE_DEFINE = 0U: MCAL APIs are executing while the CPU is in supervisory mode. (default)
- IFX_MCAL_RUN_MODE_DEFINE = 1U: MCAL APIs are executing while the CPU is in User-1 Mode without OS intervention for updating SV mode protected registers.
- IFX_MCAL_RUN_MODE_DEFINE = 2U: MCAL APIs are executing while the CPU is in User-1 Mode with OS intervention for updating SV mode protected registers.

3 New Implementation

MCAL driver APIs can be executed when the CPU is in either User-0 or User-1 or Supervisor Mode. This can be configured via precompile switches which can be configured through tresos.

Every driver in MCAL can be configured separately

- Configuration of protected mode is done in driver configuration file
- Every driver has its own header file to support this implementation(<MOD>_Protect.h)
- All operations are being separated for Initialisation, Runtime and De initialisation. (All operations other than Initialisation and De Initialisation shall come under Runtime category).

On Tresos tool

For each module in the GUI, check box switches as shown below are created based on the applicability.

1. <Mod>UserModeInitApiEnable (This switch enables or disables support for execution of Init functions in user mode)
2. <Mod>UserModeDeInitApiEnable (This switch enables or disables support for execution of De Init functions in user mode)
3. <Mod>UserModeRuntimeApiEnable (This switch enables or disables support for execution of Runtime functions(those other than Init and DeInit) in user mode)
4. <Mod>RunningInUser0Mode (This switch selects between user 0 mode and user 1 mode)

Default selection is OFF. If RunningInUser0Mode is ON then at least anyone of the switches above shall be ON.

Code generation generates following macros based on the selection

<MOD>_USER_MODE_RUNTIME_API_ENABLE = STD_ON

<MOD>_USER_MODE_INIT_API_ENABLE = STD_ON

<MOD>_USER_MODE_DEINIT_API_ENABLE = STD_ON

<MOD>_RUNNING_IN_USER_0_MODE_ENABLE = STD_ON

3.1 MCAL execution in Supervisory Mode

This mode is enabled when all above macros (**3**, **1**, **2**,**4**) should be OFF. This mode does not use any system call and thus it provides backward compatibility. This mode should be used when it is intended to run MCAL APIs in supervisory mode.

- In this mode driver read SFR directly
- In this mode driver writes SFR directly
- In this mode driver modify SFR directly
- In this mode driver set endinit directly
- In this mode driver clear endinit directly
- In this mode driver set safety endinit directly
- In this mode driver clear safety endinit directly

3.2 MCAL execution in User-1/User 0 mode with OS

Each driver API's can be configured to run is either supervisory mode or in user mode. User mode can be either user 0 or user 1. This selection of user or supervisory mode is made in configuration.

The <Mod>UserModeInitApiEnable configuration switch determines if the Initialization API's run in user mode or supervisory mode. If the switch is ON then all Initialization API's will run in user mode else it will run in supervisory mode. User mode is determined by switch <Mod>RunningInUser0Mode.

The <Mod>UserModeDeInitApiEnable configuration switch determines if the De Initialization API's run in user mode or supervisory mode. If the switch is ON then all De Initialization API's will run in user mode else it will run in supervisory mode. User mode is determined by switch <Mod>RunningInUser0Mode.

The <Mod>UserModeRuntimeApiEnable configuration switch determines if all API's other than Init and DeInit APIs will run in user mode or supervisory mode. If the switch is ON then all Runtime API's will run in user mode else it will run in supervisory mode. User mode is determined by switch <Mod>RunningInUser0Mode.

The <Mod>RunningInUser0Mode configuration switch determines whether user mode supported is User 0 or User 1. If this switch is ON then the API's selected by the switches <Mod>UserModeInitApiEnable, <Mod>UserModeDeInitApiEnable and <Mod>UserModeRuntimeApiEnable will run in User 0 mode.

If this switch is OFF then the API's selected by switches <Mod>UserModeInitApiEnable, <Mod>UserModeDeInitApiEnable and <Mod>UserModeRuntimeApiEnable will run in User 1 mode.

Please note: user 0 and user 1 mode are not supported together.

Examples:

1. All switches off. Module will run in supervisory mode.
2. <Mod>UserModeInitApiEnable – OFF

   <Mod>UserModeDeInitApiEnable – OFF

   <Mod>UserModeRuntimeApiEnable – ON

   <Mod>RunningInUser0Mode – ON. Init and DeInit in supervisory mode, Runtime API's in user 0 mode.
3. <Mod>UserModeInitApiEnable – OFF

   <Mod>UserModeDeInitApiEnable – ON

   <Mod>UserModeRuntimeApiEnable – ON

   <Mod>RunningInUser0Mode – OFF. Init in supervisory mode, DeInit and Runtime API's in user 1 mode.
4. <Mod>UserModeInitApiEnable – OFF

   <Mod>UserModeDeInitApiEnable – ON

   <Mod>UserModeRuntimeApiEnable – ON

   <Mod>RunningInUser0Mode – ON. Init in supervisory mode, DeInit and Runtime API's in user 0 mode.

When CPU is in User 1 mode the MCAL driver may require an update to a SV protected register in OS environment. When the CPU is in User 0 mode all the SFR access requires update to a SV protected register in OS environment. The MCAL driver may also require to call few functions in McalLib module Like resetendinit and setendinit in SV mode with the help of OS environment Every driver has an additional header file <Mod>_Protect.h which is an interface between MCAL and OS.

Whenever there is a request for a SV mode register access a call is made to the OS. The expectation is the OS shall perform the requested operation and return the control to the driver. For example the driver may request to write a value into the register. The OS shall update the value into the SFR address as requested by the driver. Refer **Figure 1** for sequence diagram. Similarly the driver may request to get the values from a particular SFR. The OS is expected to read the value from the requested SFR address and return the value to the driver.

Similarly some functionality of McalLib may be requested by few drivers. The expectation is the OS shall call those functions in SV mode. Refer **Figure 2** for sequence diagram.

The header file <Mod>_Protect.h is a demo file. It can be modified to introduce/access OS functionality. Some sample code is provided in the driver which can be used as reference.

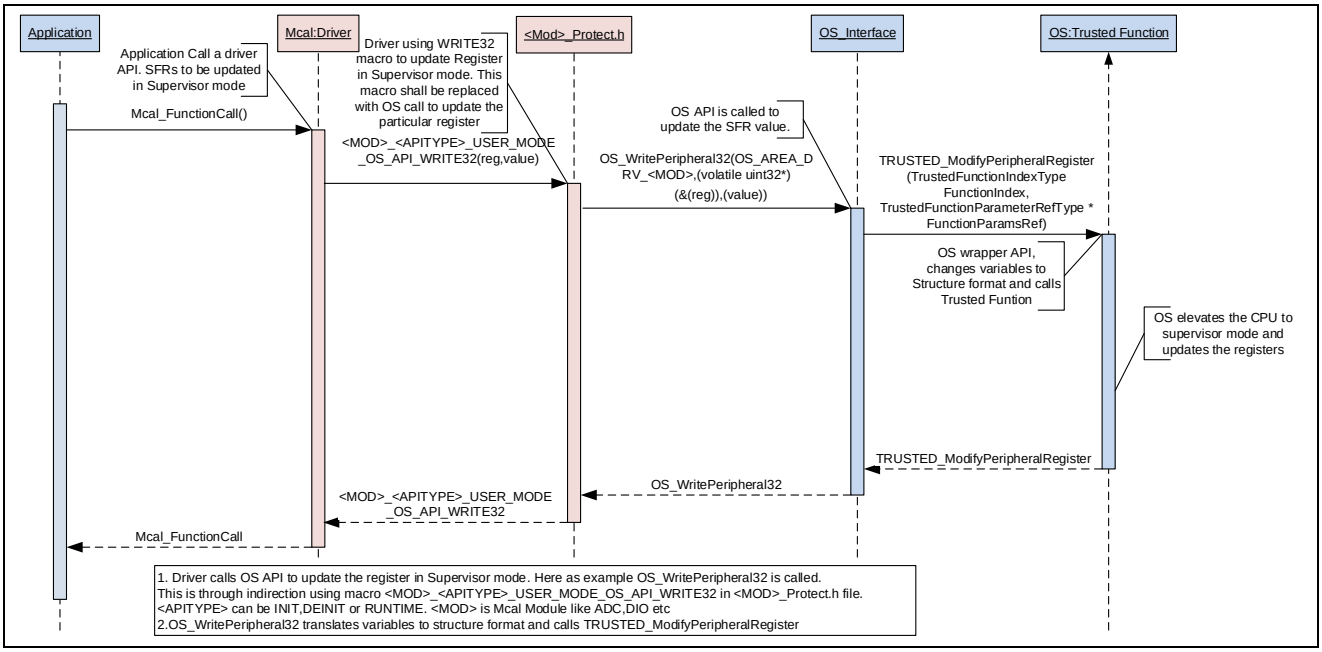

Figure 1 MCAL execution in User-1/User-0 mode with OS (SFR Write operation)

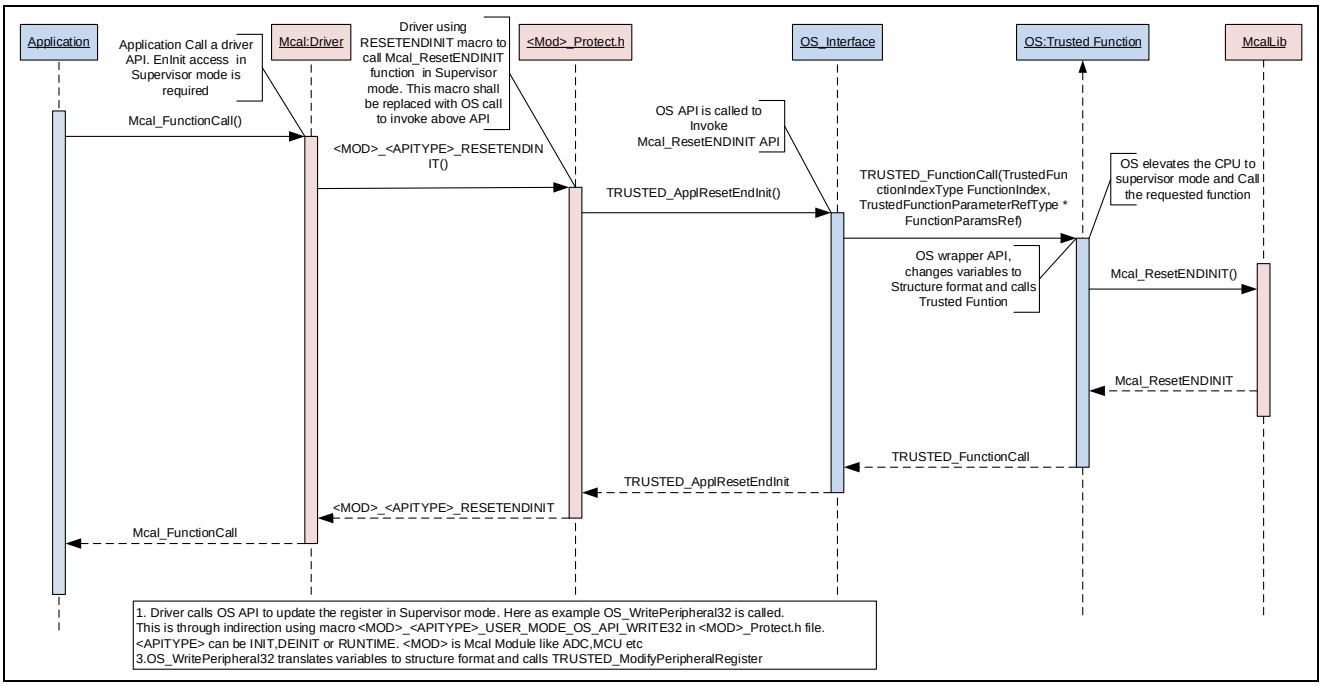

Figure 2 MCAL execution in User-1/User-0 mode with OS when McalLib API is called

4 Assumptions and conditions of use

Assumptions:

1. The following MCAL drivers do not support USR/SV mode functionality
   - FLSLOADER
   - FEE
2. For IRQ Module, Irq_<Mod>Init () functions has be executed in SV mode by customer.
3. It is assumed that the interrupt functionality for each driver will be called in SV mode only.
4. The common functions used by MCAL from the MCAL Library shall be wrapped as trusted functions (to be called by OS call)
5. <Mod>_Protect.h file shall be provided as a demo file for the customer to know the functions/defines that are implemented.
6. Customer shall implement the SchM functions, to work in User mode.

Conditions:

1. In Tresos config GUI, the following conditions have to be met
   - If <MOD>RunningInUser0Mode is set then at least any one of the below switches should be set.

     <MOD>UserModeRuntimeApiEnable

     <MOD>UserModeInitApiEnable

     <MOD>UserModeDeInitApiEnable
2. USER0 MODE – No timing requirements to customer callbacks/functions. Customer has to keep in mind that the runtime will be increased.
3. USER1 MODE - No timing requirements to customer callbacks/functions.
4. McalLib functions shall be called using trusted call methodology
   - This is a common library used between Safety Lib and MCAL.
   - This common library has functions that relate to
     - Watchdog (Mcal_WdgLib.h/.c)
     - TriCore (Mcal_TcLib.h/.c)
     - DMA (Mcal_DmaLib.h/.c)
     - Compiler related intrinsic functions (Mcal_Compiler.h/.c)
   - Since this is a common library we have to follow the “call trusted function methodology”. We designate all the McalLib functions as trusted functions. Associating of these functions shall be done by the customer in a file for e.g., OS_Stub.C. The wrappers are implemented in <Mod>.h and <Mod>_Protect.h files.
5. The invoked OS calls shall be developed by integrator according to ISO26262 based on the ASIL level of the intended application.

5 Guidelines for Usage

- The users can configure the modes for driver API's in configuration.
- If all the user mode related switches are off described in chapter 3 are OFF then the driver will work in SV mode.
- If some API's needs to be run in user mode then users can select corresponding user mode switches. Please refer section 3.2 for more details.
- When the driver is running in user mode it may SV access which needs to be facilitated by OS.
- Modify <MOD>_Protect.h and replace the stubbed calls with the actual OS calls.

  Ex: DIO driver wants to write to a register.

  The macro DIO_RUNTIME_USER_MODE_OS_API_WRITE32 should be replaced by an OS call to update the particular register. Refer **Figure 1** for sequence diagram.
- Similarly there may be some calls to McalLib functions. These functions need to be supported using trusted function method.

  E.g.: Port driver wants to call Mcal_ResetENDINIT function

  The macro PORT_INIT_RESETENDINIT() should be replace with OS functionality which will call Mcal_ResetENDINIT in SV mode. Refer **Figure 2** for sequence diagram.

w w w . i n f i n e o n . c o m

Published by Infineon Technologies AG